

BrandPulse

AI co-pilot for D2C brands. Detects wasted ad spend, explains why, recommends what to do — in plain English.

A working full-stack app: FastAPI + LangGraph + Postgres on the back end, Next.js + Tailwind on the front end. Detection runs on deterministic SQL rules. The LLM (GPT-4o-mini) only writes the narrative — it never queries the database. The whole pipeline is observable: every leak comes with a trace of which node produced what, ready to render in a "How we found this" panel.

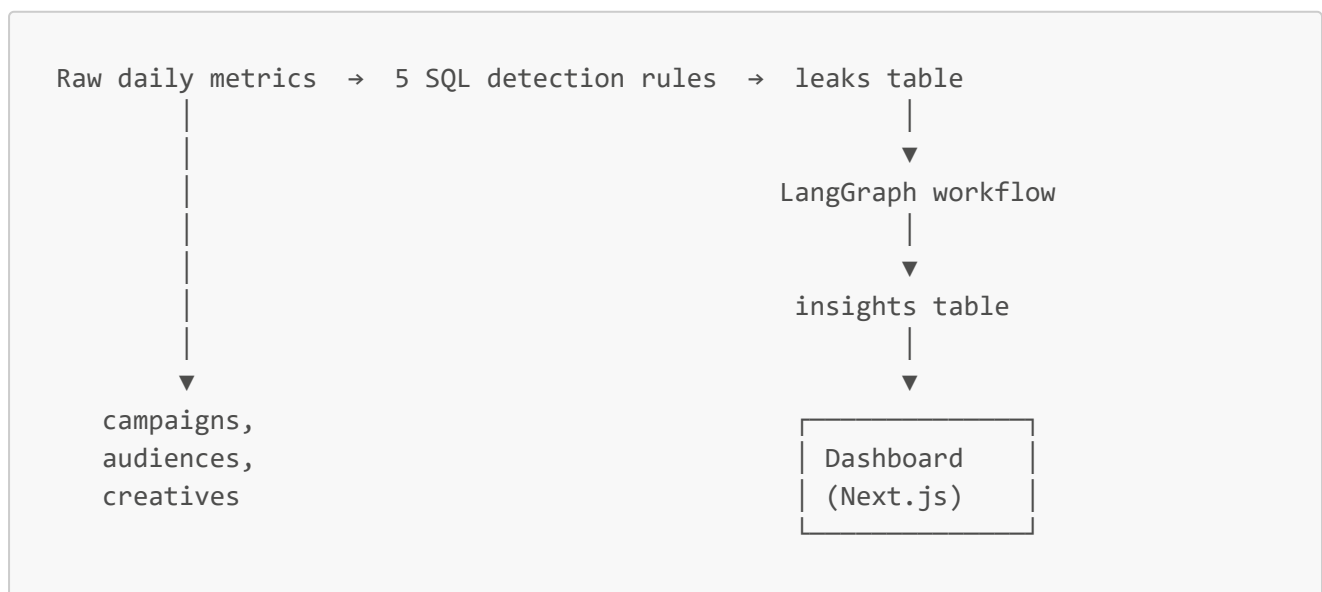
The problem

D2C founders run dozens of ad campaigns across Meta, Google, TikTok. They look at top-line metrics — total spend, blended ROAS — and miss the leaks that hide *inside* individual campaigns:

- A campaign keeps running with collapsed conversions, burning \$4k/week.
- CTR has fallen 60% over 14 days; the team doesn't notice because frequency is hidden in another tab.
- Three lookalike campaigns are quietly competing for the same audience, raising the bid for everyone.
- The bottom-quartile campaigns by ROAS absorb a quarter of total spend.

Every dashboard tool shows charts. Nobody reads them. **BrandPulse pushes one-paragraph insights** that say what's broken, why, and what to do.

What it does



The pipeline runs in four stages:

1. **Detection** — five SQL rules scan `daily_metrics` and emit `leak` rows with a dollar-impact estimate. Pure deterministic SQL, no LLM, no ML.
2. **Enrichment** — pulls extra context (brand totals, campaign details, 28-day trend, top-performers for misallocation) into a single fact pack.
3. **LLM reasoning** — GPT-4o-mini reads the fact pack and writes a cautious root-cause hypothesis with a confidence score (0–1). If confidence < 0.6, the recommender is skipped and the insight is tagged `needs_review`.
4. **Composition** — assembles the dashboard-ready card with title, summary, key facts, and recommendations.

The five leaks it detects

Leak	Fires when...	Dollar impact =
Zombie spend	Campaign still spending >\$500/week with conversions collapsed (0–1) from a healthy baseline (CVR ≥ 1%)	Last 7d spend
Creative fatigue	CTR halved (7d vs 28d), frequency > 4, creative > 14 days old	Lost conversions × historical AOV
CPA creep	CPA up 20%+ vs the 28–35-days-ago baseline, spend > \$1k/week	Extra cost per conversion × conversions in last 7d
Audience saturation	2+ active campaigns with frequency > 4 sharing audiences (same or 40%+ overlap)	Cluster spend × (1 – 4 / avg_frequency)
Budget misallocation	Bottom-quartile ROAS campaigns absorb > 25% of last-28d spend	Bottom spend × (1 – bottom_ROAS / median_ROAS)

Rule precedence: `zombie` / `fatigue` / `saturation` describe a primary root cause. CPA creep is treated as a *symptom* and is suppressed on any campaign already covered by another rule. This keeps each campaign to one primary leak instead of duplicate flags.

Tech stack

Backend

Web framework	FastAPI
Database driver	psycopg 3
Connection pool	psycopg-pool
Schema management	plain SQL files (<code>schema.sql</code>)
Orchestration	LangGraph
LLM	OpenAI GPT-4o-mini via <code>langchain-openai</code>

Output validation	Pydantic v2 (forces structured JSON from the LLM)
Database	Postgres 13+

Frontend

Framework	Next.js 16 (App Router, Server Components)
Styling	Tailwind v4
Icons	lucide-react
Charts	Server-rendered SVG (no client-side chart library)
Language	TypeScript

Infrastructure

Hosting	Render (3 services declared in <code>render.yaml</code>)
Code style	<code>ruff</code> <code>format</code> (Python), <code>prettier</code> (TypeScript)
Local DB	Local Postgres install (no Docker required)

Architecture overview

The codebase is intentionally small and lean. Each LangGraph node lives in its own file and shares state through a single TypedDict.

Backend layout

<code>schema.sql</code>	Postgres schema (9 tables)
<code>seed.py</code>	Synthetic data + 5 planted leaks
<code>detection.py</code>	5 SQL rule scanners + dollar-impact + insertion
<code>state.py</code>	LangGraph shared state (TypedDict)
<code>node_enricher.py</code>	DB context fetcher (no LLM)
<code>node_analyzer.py</code>	Cautious root-cause LLM call
<code>node_recommender.py</code>	2 next-step LLM call (skipped if confidence < 0.6)
<code>node_composer.py</code>	Final card templating
<code>workflow.py</code>	LangGraph DAG + persistence to insights table
<code>main.py</code>	FastAPI: REST endpoints + admin actions

Frontend layout

```

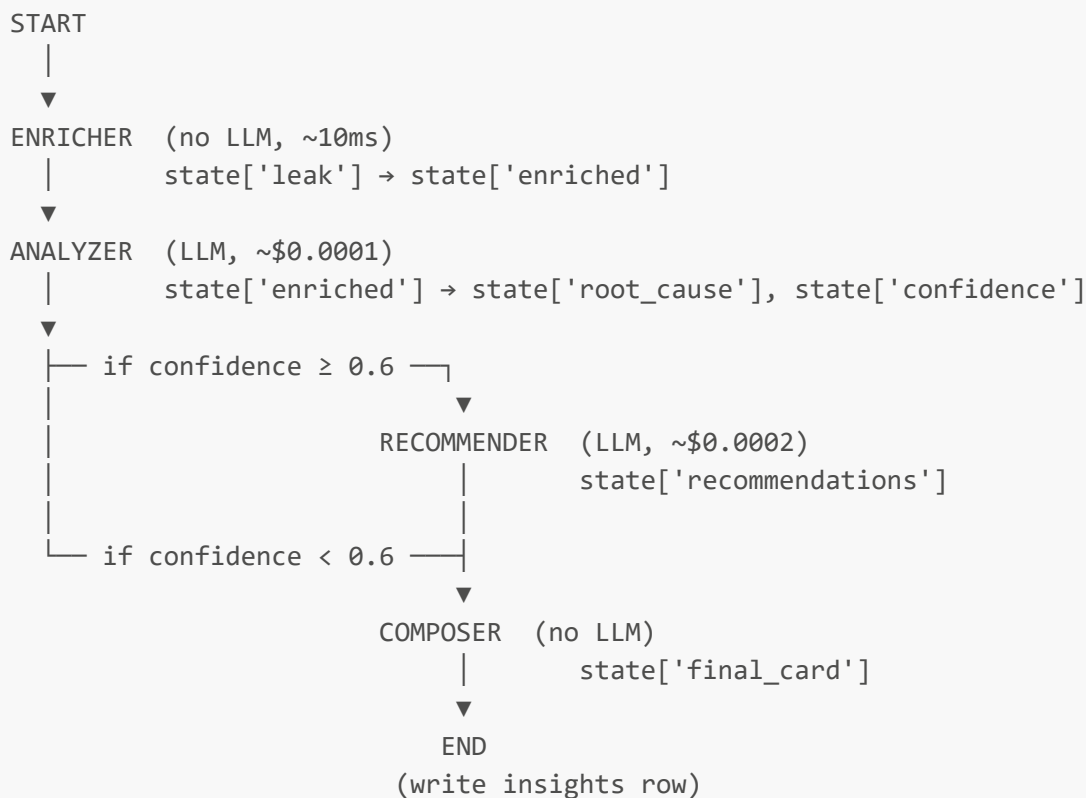
frontend/
├─ app/
│   └─ layout.tsx
└─

```

Root layout with sidebar

		page.tsx	Redirects to /dashboard
		dashboard/page.tsx	Hero + trend chart + top 3 leaks
		leaks/page.tsx	Searchable table of all leaks
		insights/[id]/page.tsx	Leak detail (chart, AI hypothesis, trace)
		campaigns/page.tsx	Table of all campaigns
		campaigns/[id]/page.tsx	Per-campaign deep dive
		connections/page.tsx	"Connected to Meta Ads" view
		components/	LeakCard, SeverityBadge, charts, etc.
		lib/	
		api.ts	TypeScript types + fetch helpers
		utils.ts	cn(), formatCurrency, formatPercent

LangGraph DAG



Every node appends one entry to `state['trace']`. That accumulating list is what powers the "How we found this" panel on the insight detail page — the user can expand it and see exactly which node did what, when, and how long it took.

Database schema

Nine tables. The relationships matter — almost every detection query starts at `daily_metrics` and joins back to `campaigns` / `audiences` / `creatives`.

```

brands
├── audiences ─── audience_overlap (pairs)

```

```

├─ creatives
├─ campaigns — audience_id, creative_id
│   └─ daily_metrics
├─ skus
├─ leaks — insights — card (JSONB), trace (JSONB), recommendations
│   └─ digests
└─ (JSONB)

```

The **leaks** and **insights** tables are split on purpose: detection is cheap and rerunnable, LLM output is expensive and cached. Re-running detection wipes leaks; re-running the workflow wipes insights for that brand only.

Synthetic data

For the demo, the seeder generates one fictional D2C brand — **Aurora Coffee Co.** — with 20 campaigns and 90 days of daily metrics. Five known leaks are planted as ground truth:

#	Leak type	Campaign
1	Zombie spend	Summer Sale - LAL 1% (last 30 days)
2	Creative fatigue	Retargeting 30d - Mug Hero (last 45 days)
3	CPA creep	Broad US - Coffee Interest (last 21 days)
4	Audience saturation	3 Cold Brew LAL campaigns sharing audiences (last 14 days)
5	Budget misallocation	6 low-ROAS campaigns absorbing ~32% of total spend

The seeder uses a fixed random seed so the data is reproducible. **seed.py** also runs verification queries at the end to confirm all 5 planted leaks are detectable.

Local development

Prerequisites

- Python 3.11+
- Node 20+
- Postgres 13+
- Poetry (for Python deps)

Setup

```

# Clone
git clone https://github.com/<your-username>/market-analyst
cd market-analyst

# Configure database connection (replace your-password)

```

```
echo "DATABASE_URL=postgresql://postgres:your-  
password@localhost:5432/market_analyst" > .env  
echo "OPENAI_API_KEY=sk-proj-..." >> .env  
  
# Install Python deps  
poetry install  
  
# Apply schema + seed data + plant leaks  
poetry run python seed.py  
  
# Run detection (5 SQL rules, no LLM)  
poetry run python detection.py  
  
# Run LangGraph workflow (~$0.001 in OpenAI)  
poetry run python workflow.py  
  
# Start backend  
poetry run uvicorn main:app --reload  
# → http://localhost:8000/docs  
  
# In a second terminal: start frontend  
cd frontend  
npm install  
npm run dev  
# → http://localhost:3000
```